

COSC 39: Theory of Computation

Problem 1.

Imagine we want to describe a deck of Poker cards using regular expressions. Here a Poker deck may have card repetition and more than 52 cards. We assume that *there are no Jokers*; every card is of one of the four suits: spades ♠, hearts ♥, diamonds ♦, and clubs ♣. We say that spades and clubs are of the color *black*, while hearts and diamonds are *red*.

Design a regular expression for the following collection of decks, as a language over alphabet $\Sigma = \{\spadesuit, \heartsuit, \diamondsuit, \clubsuit\}$:

- The collection of all Poker decks where no three cards of the same color occur consecutively.

(No formal proofs required, but explanation of your construction in a few English sentences would be helpful.)

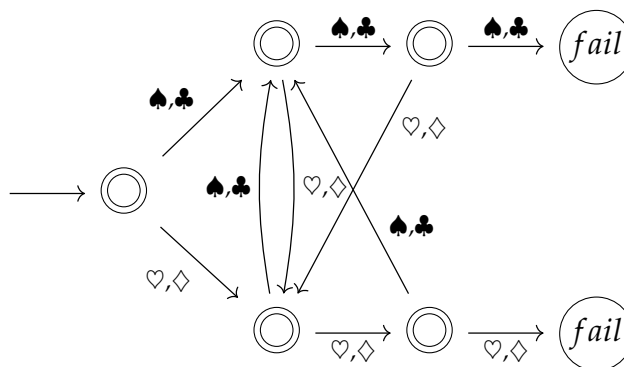
Solution.

$$(\epsilon + (\heartsuit + \diamondsuit) + (\heartsuit + \diamondsuit)^2)((\spadesuit + \clubsuit) + (\spadesuit + \clubsuit)^2)((\heartsuit + \diamondsuit) + (\heartsuit + \diamondsuit)^2)^*(\epsilon + (\spadesuit + \clubsuit) + (\spadesuit + \clubsuit)^2)$$

Problem 2.

Construct a deterministic finite automata (DFA) for the language in Problem 1.

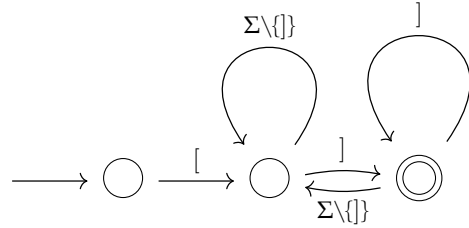
Solution.



Problem 3.

Let Σ be the alphabet $\{0, 1, [,]\}$. Define L to be the set of strings over Σ that start with $[$ and end with $]$. Notice that we allow extra $[$ and $]$ showing up within the string. For example, $[10][01]$ and $[]$ are valid strings in L , while $[010]00$ and $][]$ are not in L . Design an NFA to recognize language L .

Solution.

**Problem 4.**

Let L be an automatic language. Construct an NFA that recognize the following language.

$$L' := \{w \in \Sigma^* : \text{no proper prefix of } w \text{ is in } L\}$$

(If we use the language L from Question 1, then L' is the set of strings that start with $[$ and end with the first $]$. Your construction should work for general L .)

Solution.

Let D be a DFA that recognize the automatic language L , described by the tuple $(Q, s, A, \Sigma, \delta)$. We now construct a new ε -DFA N that recognizes the language $\text{noprefix}(L)$ as follows:

- $Q_N = Q$.
- Starting State: $S' = \{s\}$
- $A_N = A$.
- For every transition $\delta_D(q, a) = q'$ in D we add a transition $\delta_N(q', a) = q$ in N . And we remove every transition from p to other states q (even if $p = q$) if p is an accepting state in D . That is,

$$\delta(p, a) = \emptyset$$

for every $p \in A$ and $a \in \Sigma$.

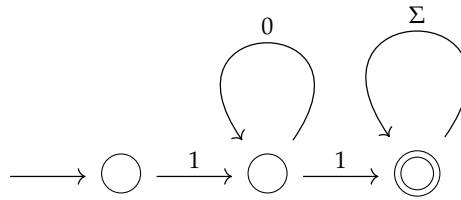
Problem 5.

Define

$$L^{\geq} := \{1^k y : y \in \{0, 1\}^* \text{ and } y \text{ contains at least } k \text{ 1s for any } k \geq 1\}$$

Draw a DFA recognizing $L^{\geq}$ with minimum number of states, and prove that it is indeed minimum by constructing a fooling set of same number of prefixes. (No need to provide distinguishing suffixes that certify the fooling set.)

Solution.



Define a fooling set $F := \{\epsilon, 0, 1\}$ with the distinguishing suffix 1.

Problem 6.

Define

$$L^{\leq} := \{1^k y : y \in \{0, 1\}^* \text{ and } y \text{ contains at most } k \text{ 1s for any } k \geq 1\}$$

Prove that $L^{\leq}$ is not regular by constructing a fooling set of infinite size. (You need to prove that it is indeed a fooling set.)

Solution.

Define the fooling set as $F := \{1^m 0 \mid m \in \mathbb{N}\}$. Then for two arbitrary prefixes $1^i 0, 1^j 0 \in F$ with $i < j$, let the distinguishing suffix be 1^j . Then we have,

- $1^i 0 1^j \notin L$, because i is smaller than j and hence y has more than i 1s.
- $1^j 0 1^j \in L$, because y has j 1s.

This implies that F is a fooling set of infinite size, and thus L is not regular.

Problem 7.

Describe a Turing machine that achieves the following task.

- On input 1^n , leave $\#^{2^n}$ somewhere on the tape.

Solution.

Problem 8.

Argue that the following Turing machine model is equivalent in power to the standard Turing machine with a single tape:

- A Turing machine that never moves its cursor to the left 3 times in a row.

Solution.

Problem 9. Prove that there is no Turing machine that decides the language

$$\text{INFINITE} := \{\langle M \rangle : L(M) \text{ contains infinitely many strings}\}$$

[Hint: You may either derive a contradiction directly, or perform a reduction from an existing uncomputable problem.]

Problem 10. Prove that the following problem is in NP:

GRAPHISO

- **Input:** Two undirected graphs G and G' .
- **Output:** Is there a mapping ϕ from V_G to $V_{G'}$ such that

$$xy \in E_G \quad \text{if and only if} \quad \phi(x)\phi(y) \in E_{G'}?$$

Problem 11. Describe a polynomial-time self-reduction for the following problem, assuming an oracle to the decision problem in Question 1.

FINDGRAPHISO

- **Input:** Two undirected graphs G and G' .
- **Output:** Compute a mapping ϕ from V_G to $V_{G'}$ such that

$$xy \in E_G \quad \text{if and only if} \quad \phi(x)\phi(y) \in E_{G'}.$$

Solution. Let $\text{ISO}(A, B)$ be the decision oracle that returns YES iff $A \cong B$. Let $n = |V_G|$; if $|V_G| \neq |V_{G'}|$ or $\text{ISO}(G, G') = \text{NO}$, report that no isomorphism exists. Otherwise we build ϕ vertex by vertex.

Pinning gadget. For graphs we have committed a partial correspondence $u_1 \mapsto w_1, \dots, u_k \mapsto w_k$ (the $u_j \in V_G, w_j \in V_{G'}$ distinct), define augmented graphs $\widehat{G}, \widehat{G}'$: for each $j = 1, \dots, k$, attach to u_j in $\widehat{G}$, and to w_j in $\widehat{G}'$, a fresh clique on $n + j$ vertices joined to that vertex by a single *bridge* edge.

Lemma. $\widehat{G} \cong \widehat{G}'$ iff there is an isomorphism $\psi : G \rightarrow G'$ with $\psi(u_j) = w_j$ for all j .

Proof. ($\Leftarrow$) Extend ψ by mapping the size- $(n + j)$ gadget on u_j isomorphically onto the size- $(n + j)$ gadget on w_j (matching bridge to bridge); this is an isomorphism $\widehat{G} \rightarrow \widehat{G}'$.

($\Rightarrow$) Each attached clique has size $n + j > n$, and the original graphs (on n vertices, with the gadgets joined only by bridges) contain no clique larger than n . So the cliques of size $n + j$ are exactly the gadgets, and their sizes are distinct, so any isomorphism $\hat{\phi} : \hat{G} \rightarrow \hat{G}'$ maps the size- $(n + j)$ gadget of $\hat{G}$ onto that of $\hat{G}'$. Within a gadget the bridge vertex is the unique one with a neighbour outside the gadget, so $\hat{\phi}$ sends u_j 's gadget-bridge to w_j 's, hence $\hat{\phi}(u_j) = w_j$. As $\hat{\phi}$ maps gadget vertices to gadget vertices, its restriction to V_G is an isomorphism $\psi : G \rightarrow G'$ with $\psi(u_j) = w_j$. $\square$

Algorithm. Fix an ordering $u_1, \dots, u_n$ of V_G . Maintain a partial map π and the set used of images taken. For $i = 1, \dots, n$:

For each candidate $v' \in V_{G'} \setminus \text{used}$: build $\hat{G}, \hat{G}'$ for the commitments $\pi(u_1), \dots, \pi(u_{i-1})$ together with the tentative pair $u_i \mapsto v'$, and call $\text{ISO}(\hat{G}, \hat{G}')$. At the first YES, set $\pi(u_i) := v'$, add v' to used, and proceed to u_{i+1} .

Output $\phi := \pi$ after all n vertices are assigned.

Correctness. Invariant: the committed partial map extends to some isomorphism $G \rightarrow G'$. It holds initially because $\text{ISO}(G, G') = \text{YES}$. Suppose it holds before processing u_i , witnessed by an isomorphism ψ extending the commitments. Then $v' = \psi(u_i)$ is an untaken candidate for which, by the Lemma, the oracle answers YES; so the loop finds some valid v' and the invariant is preserved (again by the Lemma, a YES answer means the extended commitment is realizable). After all n steps, π is a total commitment that extends to an isomorphism, i.e. π itself is an isomorphism $G \rightarrow G'$, so $xy \in E_G \iff \phi(x)\phi(y) \in E_{G'}$.

Running time. There are at most n candidates per vertex, so $O(n^2)$ oracle calls. Each augmented graph has $O(n + \sum_{j \leq n} (n + j)) = O(n^2)$ vertices and is built in polynomial time. Hence the whole self-reduction runs in polynomial time given the oracle. $\square$

Problem 12. We want to prove the following problem to be NP-hard by reduction from some known NP-hard problem:

SpanTree39

- **Input:** An undirected graph G .
- **Output:** Does G have a spanning tree with at most 39 leaves?

Why doesn't the following reduction work?

We will reduce the Hamiltonian path problem on undirected graphs, denoted HAMPATH, to SPANTREE39. Given an undirected graph G , we modify the graph by adding 38 special vertices, and connect every special vertex to an arbitrary vertex u in G , to obtain a new graph G' . The construction can be carried out in polynomial time.

To prove yes instances are preserved, assume G contains a Hamiltonian path P starting at u . Now include all special vertices and their incident edges into P to form a spanning tree T of G' . T has at most $2 - 1 + 38 = 39$ leaves.

To prove no instances are preserved, assume G' contains a spanning tree T with at most 39 leaves. Because T is spanning, all special vertices are in T . Remove all special vertices; the remaining subgraph is still a spanning tree in G and has at most 2 leaves, and therefore must be a Hamiltonian path in G .

Problem 13. Prove that SPANTREE39 is NP-hard by modifying the above construction.